

# Deferred Asynchronous Post-Quantum Signature Verification for Low-Latency TLS 1.3 Handshakes

Tahera Begum Abdul<sup>a,\*</sup>, K. Venkata Ramana<sup>a</sup>

<sup>a</sup>*Department of Computer Science and Systems Engineering, Andhra University, Visakhapatnam, Andhra Pradesh, India*

---

## Abstract

Migrating TLS 1.3 server authentication to post-quantum (PQ) signatures is urgent not because recorded traffic can be decrypted later—that threat is addressed by hybrid key exchange—but because public-key infrastructures take many years to migrate and must be quantum-safe *before* a cryptographically relevant quantum computer can forge classical signatures in real time. During the transition, many deployments are expected to use hybrid authentication, in which a classical (ECDSA) and a PQ (ML-DSA or FN-DSA) signature must both verify. This paper studies one component of the resulting handshake cost: the client-side verification of the PQ signature component.

We present *Deferred Asynchronous PQ Verification* (DAPV), a client-side, implementation-level change to the TLS 1.3 handshake state machine that requires no change to the wire format or to the server. The classical component is verified synchronously and the handshake completes in a PROVISIONAL state; the PQ component, already bound into the handshake transcript, is verified by a background worker, after which the session is promoted to FULLY\_AUTHENTICATED or terminated. We specify the semantics of the provisional state (permitted data, session tickets, alerts, and an application-facing API), implement a proof of concept in OpenSSL 3.2 with the Open Quantum Safe provider, and evaluate it for P-256 + ML-DSA-44 and P-256 + FN-DSA-512. In our single-host testbed DAPV lowers client-observed handshake latency by an RTT-independent offset of about 14–21 ms and raises the completed-handshake rate by 31–36%. We show, however, that the arithmetic of PQ verification costs below 0.15 ms, so the measured gain is attributable to non-arithmetic costs of the synchronous verification path of the prototype rather than to the cryptography itself; we discuss what this implies for when deferral is worthwhile (long certificate chains, multi-algorithm hybrids, slower schemes and constrained devices). Finally, we bound the data that can be exposed during the provisional window, identify a denial-of-service vector specific to deferred verification, and give mitigations.

**Keywords:** Post-quantum cryptography, TLS 1.3, Hybrid authentication, Deferred verification, ML-DSA, FN-DSA, Handshake latency

---

## 1. Introduction

### 1.1. Why post-quantum authentication, and why now

The public-key algorithms used by TLS today—ECDH/X25519 for key exchange and RSA or ECDSA for authentication—rely on the hardness of integer factorisation and discrete logarithms, both of which are solved efficiently by Shor’s algorithm [1] on a sufficiently large quantum computer. The two uses of public-key cryptography in TLS, however, face quantum threats of a different nature and urgency.

*Confidentiality* is threatened today by the “harvest now, decrypt later” (HNDL) strategy [2]: an adversary records encrypted sessions now and recovers their key-exchange secrets once a cryptographically relevant quantum computer (CRQC) exists. HNDL is countered by post-quantum or hybrid key exchange such as X25519MLKEM768 [3, 4], which major browsers and content delivery networks have already deployed [5]. *Authentication* is not vulnerable to HNDL: a signature that was verified in a past handshake cannot be retroactively forged to affect that handshake. The quantum threat to authentication is instead *active* impersonation once a CRQC can forge server signatures in real time. Why, then, migrate authentication now? Because the systems that carry authentication—certificate authorities, root programmes, hardware security modules, device firmware with pinned trust anchors, and certificates with multi-year validity—take many years to change. By Mosca’s inequality [6], if the migration time plus the lifetime of deployed trust anchors exceeds the time until a CRQC appears, migration must begin now. National agencies have therefore published timelines covering both key establishment and signatures [7, 8, 9], following NIST’s standardisation of ML-DSA [10], SLH-DSA [11] and the forthcoming FN-DSA [12].

---

\*Corresponding author

Email address: [taherabegum.abdul@gmail.com](mailto:taherabegum.abdul@gmail.com) (Tahera Begum Abdul)

This paper is concerned only with the authentication side. We state this explicitly because the two problems are often conflated: nothing in this paper mitigates HNDL, and our experiments are therefore designed so that the key-exchange configuration is held fixed across all measured variants (Section 5.4).

### 1.2. Hybrid authentication and its costs

During the transition, a server may authenticate with a classical signature, a PQ signature, or both. *Hybrid* authentication—in which the client accepts the server only if both a classical and a PQ signature verify—is recommended by several agencies as a hedge: the PQ schemes are young, and some candidates (e.g., Rainbow and SIKE) were broken late in the NIST process, while ECDSA remains secure against classical adversaries. Under an AND-combination, an attacker must break *both* components [13, 14]. Hybrid authentication is thus a security measure; it is *not* an interoperability mechanism. A hybrid credential in a single certificate (a *composite* key [15]) cannot be verified by a client that does not understand the composite algorithm; interoperability with such clients requires that the server select among several certificate chains using the `signature_algorithms` extension, or that the certificate carry the PQ key in a non-critical extension that legacy clients ignore [13]. Section 2.2 reviews these options and explains which of them DAPV applies to.

The cost of PQ authentication is dominated by size. An ML-DSA-44 public key and signature are 1312 and 2420 bytes, respectively, against 65 and at most 72 bytes for ECDSA P-256 (Table 1). A hybrid server flight carries at least two PQ signatures (one in the leaf certificate and one in `CertificateVerify`) plus one PQ public key; with an intermediate certificate, it carries one more of each. When the flight exceeds the TCP initial congestion window (typically ten segments, about 14.6 KB [16]), an additional round trip is required, which is the principal cause of the latency increases and throughput reductions reported in the literature [17, 18, 19, 20]. Size-reduction techniques (certificate compression, intermediate-certificate suppression [21, 22], mixed certificate chains [23], KEM-based authentication [24, 25]) address that component.

The second, smaller, component is computation. A hybrid client performs one classical and one PQ verification for `CertificateVerify`, and one of each for every certificate signature in the chain, all synchronously on the handshake critical path. DAPV targets this component *only*: it does not reduce the number of bytes on the wire and does not remove any fragmentation- or congestion-window-induced round trip.

### 1.3. Research question and approach

PQ signature verification is fast on processors with vector extensions—our own measurements give 0.08–0.14 ms per ML-DSA-44 or FN-DSA-512 verification with AVX2 (Section 6.5). The question we ask is therefore not “is PQ verification slow?” but: *if the PQ component of a hybrid credential is verified after, rather than before, the handshake completes, what latency and throughput changes are observed in a real TLS stack, where do they come from, and what security exposure does the deferral create?* DAPV answers this by verifying the classical component synchronously (which yields ordinary classical authentication immediately) and verifying the PQ component in a background worker, while the handshake transcript—which already contains the PQ signature bytes—binds the deferred check to the session.

### 1.4. Contributions

We deliberately position DAPV as an *implementation-level* scheduling strategy with precisely specified local state semantics, not as a new TLS protocol mechanism: it changes neither messages nor the wire format, and a DAPV client interoperates with unmodified servers. Our contributions are:

- (1) **Provisional-state semantics.** A specification of the client state machine with a PROVISIONAL state: entry and promotion conditions, the behaviour on failure (alert and teardown), handling of session tickets, resumption and 0-RTT, and an application-facing API to enforce a data cap (Sections 4 and 7.4).
- (2) **Prototype.** An implementation in OpenSSL 3.2 with the OQS provider on Windows 11 and Ubuntu 22.04 (Section 5.1); **[AUTHOR INPUT: source code URL / archive DOI, see Section 5.2]**.
- (3) **Measurement and decomposition.** Latency, completed-handshake rate, CPU utilisation and provisional-window duration for two hybrid configurations, five RTTs and three concurrency levels, including a decomposition showing that the measured gain is not explained by the cost of PQ arithmetic (Sections 6.1–6.5).
- (4) **Risk characterisation.** A bound on data exposed during the provisional window that accounts for both link rate and TCP congestion window, a DAPV-specific denial-of-service vector, and mitigations (Section 7).

Table 1: Sizes (bytes) of the objects that appear in a TLS 1.3 handshake for the algorithms discussed. Values from the respective standards.

| Algorithm                     | Security category | Public key          | Signature / ciphertext |
|-------------------------------|-------------------|---------------------|------------------------|
| ECDSA P-256                   | classical         | 65                  | $\leq 72$              |
| FN-DSA-512 (Falcon)           | 1                 | 897                 | $\approx 666$          |
| ML-DSA-44                     | 2                 | 1312                | 2420                   |
| ML-DSA-65                     | 3                 | 1952                | 3309                   |
| SLH-DSA-SHA2-128s             | 1                 | 32                  | 7856                   |
| X25519MLKEM768 (key exchange) | 3                 | 1216 (client share) | 1120 (server share)    |

### 1.5. Scope and limitations

DAPV requires a classical component to provide the synchronous anchor for the provisional state; PQ-only deployments are out of scope. During the provisional window the session has classical-strength authentication only, so DAPV is unsuitable wherever even a bounded interval of classical-only authentication is unacceptable. Our testbed places client and server on the same host **[AUTHOR INPUT: confirm]**; the implications are discussed in Section 6.7.

Section 2 gives the background on PQ signatures, hybrid constructions and TLS 1.3 authentication, and Section 3 reviews related work. Section 4 presents the design, Section 5 the experimental setup and Section 6 the results. Section 7 analyses security, Section 8 discusses deployment and Section 10 concludes.

## 2. Background

### 2.1. Post-quantum signature schemes

A signature scheme is *post-quantum* if its security rests on problems for which no efficient quantum algorithm is known, as opposed to factoring and discrete logarithms. NIST has standardised three families:

- **ML-DSA** (FIPS 204, formerly CRYSTALS-Dilithium) [10]: a Fiat–Shamir-with-aborts scheme based on the Module-LWE and Module-SIS lattice problems. Signing and verification use only integer arithmetic and the NTT, are simple to implement in constant time and vectorise well (AVX2 implementations are part of the reference package [26]). Signatures and keys are comparatively large.
- **FN-DSA** (FIPS 206, formerly Falcon; draft at the time of our experiments) [12]: a hash-and-sign scheme over NTRU lattices. It has the smallest combined public key and signature of the standardised schemes and fast verification, but signing requires floating-point Gaussian sampling, which complicates constant-time implementation.
- **SLH-DSA** (FIPS 205, formerly SPHINCS+) [11]: a stateless hash-based scheme whose security relies only on properties of the hash function. It has tiny public keys but signatures of 7.8–50 KB and markedly slower signing and verification.

Choosing among them is a trade-off between size (which drives network cost), verification speed (which drives client CPU cost), signing speed and implementation risk (which drive server cost and side-channel exposure), and the conservativeness of the security assumption. For TLS server authentication, where every handshake transmits the leaf public key and at least two signatures, ML-DSA-44 and FN-DSA-512 are the most commonly evaluated options; we use both. Table 1 lists the object sizes relevant to this paper.

### 2.2. Hybrid signature constructions

A hybrid (or *multi-algorithm*) authentication combines a classical and a PQ signature so that security holds while at least one of them is unbroken [13, 14, 27]. Four constructions are relevant to TLS:

- Concatenation / composite.** A single composite public key and a signature  $\sigma = \sigma_E \parallel \sigma_P$  under a new algorithm identifier, as in the IETF LAMPS composite ML-DSA specification [15]. Simple, but not backward compatible: a client that does not recognise the composite algorithm cannot verify it. Without further measures, a concatenation is *separable*—an attacker can strip one component and present the other as a stand-alone signature—which composite specifications prevent by binding the algorithm identifier into the signed message [14, 15].
- Nested / alternative-signature extensions.** A classical certificate that additionally carries a PQ key and PQ signature in non-critical X.509 extensions, so that legacy verifiers ignore them [13]. Backward compatible, at the cost of carrying both sets of objects to every client.

- (c) **Parallel certificate chains.** The server holds a classical and a PQ chain and selects one per client using `signature_algorithms`. Interoperable, but each client verifies only one algorithm, so it is not hybrid in the AND sense.
- (d) **Mixed chains.** Different algorithms at different levels of the chain [23], e.g., a PQ root and intermediate with a classical leaf, trading size against the level at which quantum resistance is achieved.

DAPV applies whenever a single client must verify both a classical and a PQ signature on the same handshake, i.e., to (a) and (b), and to the PQ-signed levels of (d). It does not apply to (c). Our prototype uses a concatenation encoding (Section 5.1); it is not claimed to satisfy the non-separability properties of the composite specification [15], which a production deployment should adopt. We note that DAPV’s own design makes stripping of  $\sigma_P$  yield at most a provisional session, because the client never promotes a session without a valid  $\sigma_P$ .

### 2.3. TLS 1.3 authentication

TLS 1.3 [28] supports three ways of authenticating a peer.

- (i) **Certificate-based authentication** in a full handshake. After `ServerHello` establishes handshake traffic keys, the server sends, encrypted, `EncryptedExtensions`, `Certificate` (its chain), `CertificateVerify` and `Finished`. The client validates the chain to a trust anchor, which requires verifying the signature on each certificate issued by a CA in the chain, and verifies `CertificateVerify`: a signature, under the leaf key, over 64 bytes of 0x20, the context string “TLS 1.3, server `CertificateVerify`”, a zero byte and the transcript hash of all handshake messages up to and including `Certificate`. The server `Finished` is an HMAC over the transcript up to and including `CertificateVerify`; the client `Finished` is an HMAC over the transcript up to and including the server `Finished`. A failed `CertificateVerify` check must terminate the handshake with a `decrypt_error` alert; an invalid chain with `bad_certificate` or a related alert.
- (ii) **PSK-based authentication** (session resumption or an external PSK). No certificate or signature is exchanged; authentication is inherited from the session that issued the ticket (or from the external key). With 0-RTT, the client may send early data under the PSK before the server’s first flight; such data is not forward secret and is replayable.
- (iii) **Post-handshake client authentication**, in which the server requests a client certificate after the handshake.

The client may also be authenticated with a certificate during the main handshake (mutual TLS), in which case the *server* performs the verification. DAPV applies to certificate-based authentication in full handshakes (i). For (ii) there is no signature to defer, but resumption must be restricted so that it does not inherit a provisional authentication (Section 4.3). In mutual TLS, the same technique could be applied by the server to the client’s hybrid signature; we do not evaluate this case.

The prototype applies deferral to **[AUTHOR INPUT: state precisely which signatures are deferred: only the PQ component of `CertificateVerify`, or also the PQ components of certificate signatures in the chain]**, and the chain in all experiments consists of **[AUTHOR INPUT: number of certificates sent by the server, and whether the leaf is self-signed or issued by a test CA]**.

## 3. Related Work

*PQ performance in TLS.* Paquin, Stebila and Tamvada [19] benchmarked PQ key exchange and signatures in TLS under emulated network conditions and showed that the effect of size dominates that of computation as packet loss and RTT grow. Sikeridis, Kampanakis and Devetsikiotis [17] studied PQ authentication in TLS 1.3 specifically, showing that the added round trip caused by exceeding the TCP initial congestion window is the main source of slowdown and that verification cost is small; they later extended the analysis to TLS and SSH [18]. Sosnowski et al. [29] and Montenegro et al. [20] report evaluation frameworks and results across classical, hybrid and PQ configurations with OpenSSL and OQS. These studies measure costs; none proposes changing *when* verification is scheduled.

*Reducing authentication size.* KEMTLS replaces handshake signatures by KEM-based implicit authentication [24], with a variant using pre-distributed keys [30] and an IETF draft [25]; Gonzalez and Wiggers evaluated it on embedded devices [31]. Intermediate-certificate suppression [21, 22] and mixed chains [23] reduce the number of large objects transmitted. These approaches attack the dominant, size-related cost and are complementary to DAPV, which attacks the computational one; with them in place, the relative importance of computation increases.

*Hybrid signatures.* Bindel et al. [13] formalised hybrid signatures for PKI transition and analysed nested and concatenated combinators; Bindel and Hale [14] and the IETF hybrid-signature spectrum document [27] classify separability and backward-compatibility properties; composite ML-DSA is being standardised in the IETF LAMPS working group [15].

*Asynchronous and early-data processing in TLS.* Deferring or overlapping cryptographic work with the handshake has precedents. OpenSSL’s asynchronous mode and the QTLS framework [32] offload RSA/ECC operations to accelerators and resume the handshake on completion; this overlaps work *across* connections but keeps each connection blocked until its operation completes, whereas DAPV lets a connection proceed before its PQ verification completes. TLS False Start [33] allowed a TLS 1.2 client to send application data before receiving the server **Finished**, accepting a bounded risk in exchange for one round trip; TLS 1.3 0-RTT similarly trades replay protection for latency [28]. DAPV follows the same pattern—accept a weaker guarantee for a bounded time, under explicit restrictions on what may be sent—applied to the PQ component of authentication.

*Formal analysis of deferred authentication.* Cremers et al. [34] found an attack on an early draft of TLS 1.3 delayed client authentication that arose because the delayed signature was not bound to the full session transcript; the final TLS 1.3 design and its symbolic analysis [35] ensure such binding. In DAPV the deferred signature is part of **CertificateVerify**, which is included in the transcript authenticated by both **Finished** messages before any application data is sent (Section 4.4). A formal model of the provisional state is future work.

*Industrial deployment.* Google’s CECPQ1 and CECPQ2 experiments [36, 37, 38] and Cloudflare’s experiments [5] concerned PQ key exchange, which is now widely deployed. Deployment of PQ authentication is at an earlier stage and its size cost is the main obstacle discussed [39, 40].

## 4. DAPV Design

### 4.1. Design goals

DAPV is designed to (G1) remove the verification of the PQ component from the client’s handshake critical path; (G2) never promote a session to full authentication unless the PQ component has verified, and terminate the session if it fails; (G3) bound what the application may send before promotion; and (G4) require no change to the server, the messages or the wire format, so that a DAPV client interoperates with any server offering the same hybrid credential. Non-goals are reducing handshake size or round trips, and PQ-only deployments.

### 4.2. Handshake flow

Figure 1 shows a full TLS 1.3 handshake with DAPV on the client. The numbered steps are:

1. **ClientHello**: the client offers key-exchange groups and, in **signature\_algorithms**, the hybrid algorithm.
2. **ServerHello**: the server selects the group; both sides derive handshake traffic keys. All following server messages are encrypted.
3. **EncryptedExtensions, Certificate, CertificateVerify, Finished**: the server sends its chain and the hybrid signature  $\sigma_E \parallel \sigma_P$  over the transcript, followed by its **Finished**. The transcript hash now covers  $\sigma_P$ .
4. **Classical verification**: the client validates the chain’s classical signatures and verifies  $\sigma_E$  synchronously. On failure it sends the corresponding fatal alert and aborts; no PQ work is scheduled.
5. **Dispatch**: the client submits the job  $(pk_P, m, \sigma_P, sid)$  to a verification worker and enters PROVISIONAL. Here  $m$  is the **CertificateVerify** content defined in Section 2.3.
6. **Client Finished**: the client verifies the server **Finished**, sends its own **Finished** and derives the application traffic keys. The handshake is complete from the server’s point of view, and application data may flow subject to the policy of Section 4.3.
7. **PQ verification**: the worker verifies  $\sigma_P$ .  $\Delta t$  is the time from step 5 to the end of this step; it consists of queueing and scheduling delay plus the verification itself.
8. **Promotion or termination**: on success the session becomes FULLY\_AUTHENTICATED; on failure the client sends a fatal **decrypt\_error** alert, closes the connection, and discards all state derived from the session, including tickets (Section 4.3).

### 4.3. Provisional-state semantics

*Promotion does not re-key.* The transition from PROVISIONAL to FULLY\_AUTHENTICATED changes only a local state variable; no message is exchanged and the traffic keys are not changed. Re-keying is unnecessary because the keys depend only on the (EC)DHE/KEM shared secret and the transcript, both fixed at step 6; the PQ verification adds assurance about *who* holds those keys, not about the keys themselves. If  $\sigma_P$  verifies, the peer with whom the keys were established during  $\Delta t$  is the same peer that is now fully authenticated, so data sent during  $\Delta t$  is retroactively covered by the full hybrid guarantee. If it fails, that data may have been sent to an impostor who defeated only the classical component; this is the exposure bounded in Section 7.2. A TLS **KeyUpdate** after promotion is permissible but adds no authentication.

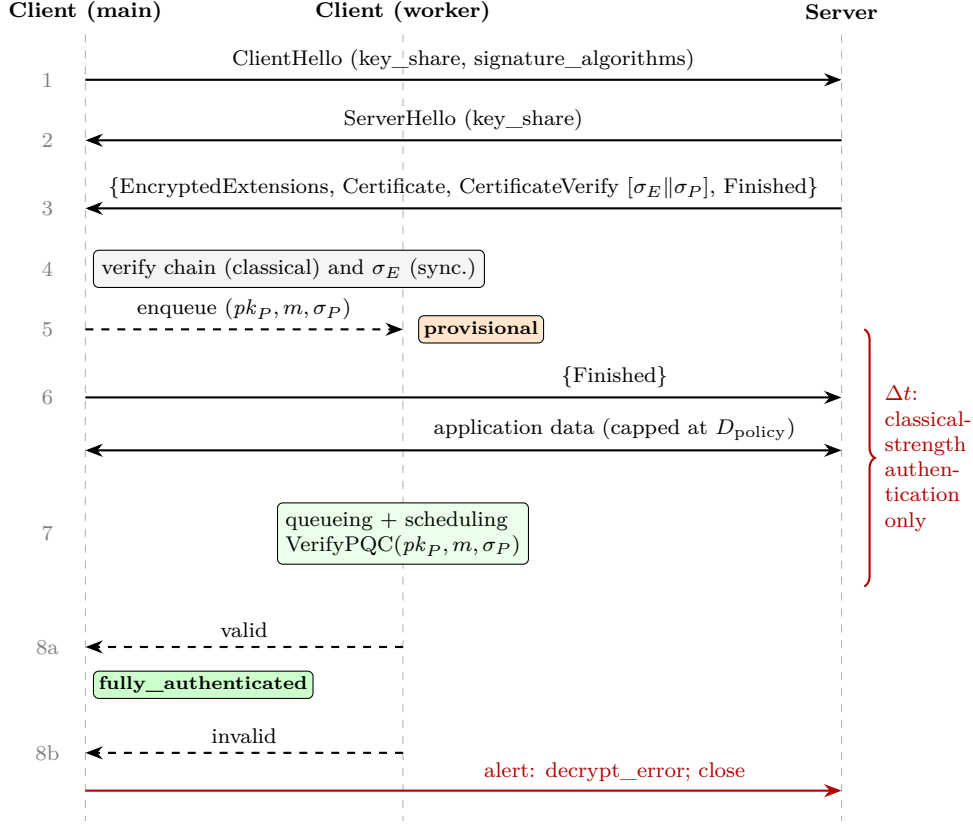


Figure 1: TLS 1.3 full handshake with DAPV on the client. Messages in braces are encrypted under handshake traffic keys.  $\sigma_P$  is part of **CertificateVerify** and hence of the transcript authenticated by both **Finished** messages (step 3 onwards), so the deferred check in step 7 necessarily concerns the signature received in this handshake. No message is added or changed; steps 4–8 are local to the client.

*Data policy.* While **PROVISIONAL**, the application may send at most  $D_{\text{policy}}$  bytes and should send only data it would be willing to reveal to a classical-only authenticated peer (Algorithm 3). Received data may be processed but should not trigger irreversible actions.

*Tickets, resumption and 0-RTT.* Because PSK resumption inherits the authentication of the issuing session, a **NewSessionTicket** received while **PROVISIONAL** must be held and made usable only on promotion, and discarded on failure. Otherwise an attacker who passed only the classical check could obtain a ticket whose resumptions—including 0-RTT early data—never undergo PQ verification.

*Failure.* On PQ failure the client sends a fatal **decrypt\_error** alert (the alert TLS 1.3 prescribes for a failed **CertificateVerify**), closes the connection, marks the peer’s credential as failed for the lifetime of the process, and reports the failure to the application.

#### 4.4. Transcript binding

The server **Finished** MAC covers **CertificateVerify** and therefore  $\sigma_P$ ; the client verifies it before step 6. An attacker therefore cannot substitute  $\sigma_P$  with a signature from another session, or modify it in transit, without causing the server **Finished** check to fail. Binding guarantees *which* signature is verified later; it does not by itself guarantee that the signature is *valid*, which is precisely what step 7 checks.

#### 4.5. Algorithms

Algorithm 1 gives the client procedure. **VerifyECDSA** and **VerifyPQC** denote the verification algorithms of ECDSA P-256 and of the PQ scheme (ML-DSA.Verify of FIPS 204 [10] or FN-DSA verification [12]), each applied to the corresponding component public key  $pk_E$ ,  $pk_P$  from the leaf certificate, the message  $m$  of Section 2.3 and the corresponding signature component; each returns a Boolean.

*Dispatch order: after, not concurrently with, the classical check.* Two reasons. First, dispatching only after  $\sigma_E$  has verified ensures that a peer who cannot produce a valid classical signature cannot cause any PQ work or occupy a worker; dispatching first would let an unauthenticated attacker consume workers at the cost of a single message, strengthening the flooding attack of Section 7.3. Second, the classical verification takes tens of microseconds, so starting the PQ job earlier would shorten  $\Delta t$  by at most that amount. Implementations that prefer to overlap the two can dispatch first and cancel the job if  $\sigma_E$  fails.

---

**Algorithm 1:** DAPV client processing of a hybrid `CertificateVerify`

---

**Input:** Leaf keys  $(pk_E, pk_P)$ ; message  $m$ ; signature  $\sigma_E \parallel \sigma_P$ ; session id  $sid$

```
1 if not VerifyECDSA( $pk_E, m, \sigma_E$ ) then
2   | send alert decrypt_error; abort // no PQ work is scheduled
3 Submit( $pk_P, m, \sigma_P, sid$ ) to the worker pool (Algorithm 2)
4  $session.state \leftarrow$  PROVISIONAL
5 verify server Finished; send client Finished; derive application traffic keys
6 on completion of job ( $sid, ok$ ):
7   | if  $ok$  then
8     |  $session(sid).state \leftarrow$  FULLY_AUTHENTICATED; release held session tickets
9   | else
10  | send alert decrypt_error; close  $session(sid)$ ; discard held tickets
```

---

---

**Algorithm 2:** Bounded verification worker pool (recommended design; the evaluated prototype uses one thread per job, Section 5.1)

---

**Input:** Pool size  $M$ ; queue capacity  $Q$

```
1 start  $M$  worker threads, each executing:
2 while running do
3   | ( $pk_P, m, \sigma_P, sid$ )  $\leftarrow$  BlockingDequeue()
4   |  $ok \leftarrow$  VerifyPQC( $pk_P, m, \sigma_P$ )
5   | notify the owning connection of ( $sid, ok$ )
6 function Submit( $pk_P, m, \sigma_P, sid$ ):
7   | if queue length  $< Q$  then
8     | enqueue the job
9   | else
10  | verify synchronously on the caller's thread // back-pressure
```

---

#### 4.6. Design rationale: threads versus instruction-level parallelism

A natural question is why DAPV uses a separate thread when PQ verification is already fast and vector instructions (AVX2, AVX-512) accelerate it further. The two techniques are complementary rather than alternatives. SIMD reduces the *duration* of one verification and is already used by our prototype (liboqs AVX2 code paths). Thread-level deferral changes *where* the remaining duration is spent: off the handshake critical path. For a single AVX2 ML-DSA-44 verification the remaining duration is about 0.1 ms, and deferring it cannot save more than that; we confirm in Section 6.2 that the latency gain we measure is *not* explained by this cost.

Deferral becomes worthwhile when the synchronous PQ work on the critical path is large, which happens when (i) the chain contains several PQ-signed certificates, possibly with PQ-signed certificate-transparency timestamps, so that a handshake needs several PQ verifications; (ii) a slower scheme such as SLH-DSA is used; (iii) the client lacks vector units (microcontrollers, low-end ARM cores); or (iv) the credential combines more than two algorithms (e.g., ECDSA + ML-DSA + SLH-DSA). Cases (i) and (iv) consist of *independent* verifications, which SIMD cannot parallelise with one another but a worker pool can, one per core. Algorithm 1 generalises directly: all PQ components are submitted as separate jobs and the session is promoted when all have succeeded. We have not evaluated these cases; they are the most promising direction for follow-up measurements **[AUTHOR INPUT: if you add experiments with 2–3-certificate chains or three-algorithm hybrids, report them here]**.

#### 4.7. Analytical model

Let  $k$  be the number of round trips of the handshake (one for a TLS 1.3 full handshake as seen by the client) and  $L_0$  the RTT-independent processing time. Then

$$L = L_0 + (k + \delta k) \text{RTT}, \quad (1)$$

where  $\delta k \geq 0$  counts additional round trips, which occur when the server's first flight exceeds the sender's congestion window (initially about ten segments [16]), *not* merely when a message spans several segments: segments within the window are sent back to back. DAPV removes the synchronous PQ-path time  $T_{\text{PQ}}$  from  $L_0$  and leaves  $\delta k$  unchanged:

$$L_{\text{DAPV}} = (L_0 - T_{\text{PQ}}) + (k + \delta k) \text{RTT}. \quad (2)$$

---

**Algorithm 3:** Application-layer data policy

---

**Input:** Session state; bytes already sent while provisional  $s$ ; payload size  $d$ ; cap  $D_{\text{policy}}$

```
1 if  $state = \text{PROVISIONAL}$  and  $(s + d > D_{\text{policy}}$  or  $payload \text{ is sensitive})$  then  
2   | buffer payload until FULLY_AUTHENTICATED (or drop on failure)  
3 else  
4   | transmit payload; if  $state = \text{PROVISIONAL}$  then  $s \leftarrow s + d$ 
```

---

Consequently the saving is an RTT-independent offset,  $T_{PQ}$ , and the curves of synchronous and DAPV latency against RTT have equal slope. For the single-certificate configuration used in our experiments the server flight is approximately 7 KB (two ML-DSA-44 signatures, one ML-DSA-44 public key, the classical components and certificate overhead), below the initial window, so we expect  $\delta k = 0$ ; our measurements (Section 6.1) agree.

## 5. Implementation and Experimental Setup

### 5.1. Implementation

The prototype is built on OpenSSL 3.2, liboqs [AUTHOR INPUT: version] and the OQS provider [AUTHOR INPUT: version] [41]. Hybrid certificates carry the concatenation of the P-256 and PQ public keys; signatures are encoded as  $\text{len}(\sigma_E) \parallel \sigma_E \parallel \sigma_P$ . We modified the provider’s hybrid signature-verification routine (`oqs_sig_verify`) so that, when invoked for the server’s `CertificateVerify`, it verifies the classical component, copies  $(pk_P, m, \sigma_P)$  into a job, dispatches it with `CreateThread` (Windows) or `pthread_create` (Linux), and returns success to OpenSSL so that the handshake proceeds. A per-connection state object, reachable through `SSL_get_ex_data`, records the state and receives the job result; on failure the main thread sends the alert at its next I/O operation [AUTHOR INPUT: confirm mechanism, e.g., polled flag or wake-up via socket pair]. The prototype creates one thread per job; the bounded pool of Algorithm 2 and the ticket handling of Section 4.3 are [AUTHOR INPUT: implemented / not implemented in the prototype—state which].

### 5.2. Code and data availability

The source code of the modified provider, the load generator, the measurement scripts and the raw measurements are available at [AUTHOR INPUT: public repository URL and archived DOI (e.g., Zenodo)].

### 5.3. Hardware and operating systems

Two x86-64 testbeds were used: an Intel Core i7-11800H (8 cores, 2.3 GHz, 32 GB RAM) running Windows 11 Pro, and an Intel Xeon Silver 4310 (12 cores, 2.1 GHz, 64 GB RAM) running Ubuntu 22.04 LTS; AVX2 was enabled on both. Compilers were Visual Studio 2022 and GCC 11.3. On each testbed, the TLS client (load generator) and server ran on the same host [AUTHOR INPUT: confirm; state CPU pinning of client and server processes, if any]. Results in Section 6 are from the [AUTHOR INPUT: Windows / Linux] testbed unless stated otherwise.

### 5.4. Cryptographic configuration

Server authentication used five configurations: ECDSA P-256 (classical baseline); ML-DSA-44 and FN-DSA-512 alone (synchronous PQ-only baselines); and the hybrids P-256 + ML-DSA-44 and P-256 + FN-DSA-512, each measured with synchronous verification and with DAPV. Key exchange used the group [AUTHOR INPUT: state the group actually negotiated, e.g., X25519 or X25519MLKEM768] in *all* configurations, so that differences between configurations are attributable to authentication alone. [AUTHOR INPUT: If the group was X25519: add “Using X25519MLKEM768 instead would add 1216 and 1120 bytes to ClientHello and ServerHello respectively and about 0.1 ms of computation to both sides, but would not interact with signature verification; the results therefore transfer to a full PQ configuration except for these additive terms.” Better: re-run the hybrid configurations with X25519MLKEM768.]

### 5.5. Network emulation

Delay was added with `Clumsy 0.3` (Windows) and `tc netem` (Linux) to obtain RTTs of 0, 20, 50, 100 and 150 ms, without packet loss. On Linux the `qdisc` was attached to [AUTHOR INPUT: interface—if the loopback interface, state how delay was split between directions].

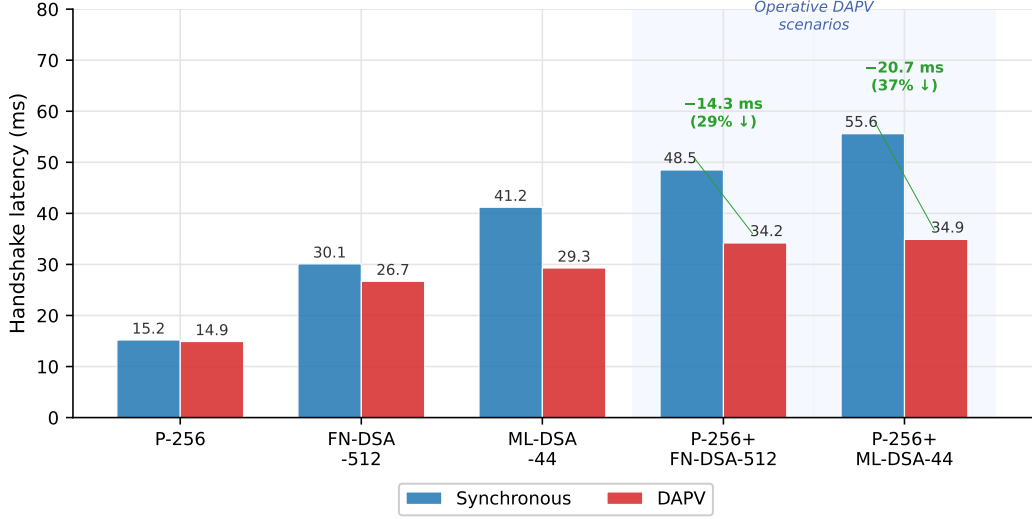


Figure 2: Handshake latency at 0 ms added RTT, synchronous (blue) versus DAPV (red). For standalone PQ configurations, the DAPV bars quantify thread overhead only; they are not operative deployments.

### 5.6. Metrics and procedure

1. **Handshake latency:** at the client, from sending `ClientHello` to the point at which the client may send application data (after sending its `Finished`), i.e., the latency seen by the application.
2. **Completed-handshake rate:** handshakes completed per second by a closed-loop load generator with 100, 500 or 1000 concurrent clients, each opening a new connection as soon as its previous handshake completes **[AUTHOR INPUT: confirm tool and closed-loop behaviour; confirm sessions are not resumed]**. Because client and server share a host, this is a system-level rate, not a server-only capacity.
3. **Peak CPU utilisation** of the host during a run.
4.  $\Delta t$ : from job submission (step 5) to completion of PQ verification (step 7), decomposed into the verification call alone, timed inside the worker, and the remainder (thread creation, queueing and scheduling).

Each configuration was measured in 30 independent runs **[AUTHOR INPUT: state how many handshakes per run]**. With the observed standard deviations of latency and throughput (below 4% of the mean), 30 runs give a 95% confidence-interval half-width of about 1.5% of the mean, which is an order of magnitude smaller than the differences we report.  $\Delta t$  is considerably more dispersed (Table 2); its tail percentiles are reported as descriptive statistics of the pooled samples.

## 6. Results

### 6.1. Handshake latency

*Takeaway:* DAPV lowers latency by an RTT-independent offset; it does not change the slope. At 0 ms RTT (Figure 2), DAPV lowers latency from 48.5 to 34.2 ms for P-256 + FN-DSA-512 (−29%) and from 55.6 to 34.9 ms for P-256 + ML-DSA-44 (−37%). Against RTT (Figure 3) the classical, synchronous-hybrid and DAPV curves all have a slope of 1.0 ms per ms of RTT. For P-256 + ML-DSA-44 the DAPV saving is 20.7 ms at 0 ms RTT and 21.0 ms at 150 ms, i.e., RTT-independent to within measurement precision. For P-256 + FN-DSA-512 the saving is 14.3 ms at 0 ms and 20.0 ms at 150 ms RTT **[AUTHOR INPUT: this growth is not predicted by Equation (2); please check both values against the raw data and explain or correct]**. Equal slopes mean  $\delta k = 0$  in our configuration, as predicted in Section 4.7. The relative saving therefore falls with RTT—from 37% to 10% for P-256 + ML-DSA-44—because the absolute saving is fixed while the RTT term grows.

### 6.2. Where does the latency saving come from?

*Takeaway:* the saving is two orders of magnitude larger than the cost of PQ arithmetic. Timed in isolation, a PQ verification takes 0.08–0.14 ms (Table 2), yet DAPV saves 14–21 ms. The saving must therefore come from other work on the synchronous verification path of the prototype, which in the synchronous build runs inline on the handshake thread and in the DAPV build runs on the worker. Candidates are per-call provider dispatch and key decoding, memory allocation, and waiting that is quantised by the operating-system timer: the default timer period on Windows is 15.6 ms, close to both the saving and the scheduling component of  $\Delta t$ . **[AUTHOR INPUT: Profile the synchronous verification path (e.g., perf, VTune or ETW) and report the breakdown**

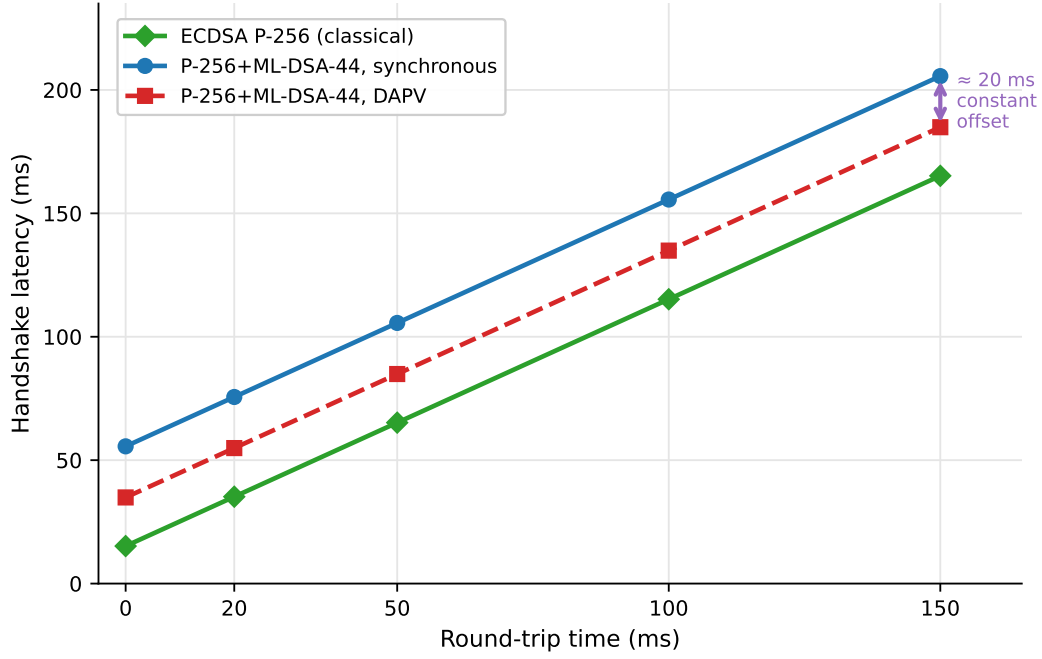


Figure 3: Handshake latency versus RTT for P-256 + ML-DSA-44. The three curves have the same slope; DAPV is offset by  $\approx 20$  ms from synchronous hybrid at every RTT. **[AUTHOR INPUT: Confirm that the points at RTT = 20, 50 and 100 ms are measured. If they were interpolated, remove those markers.]**

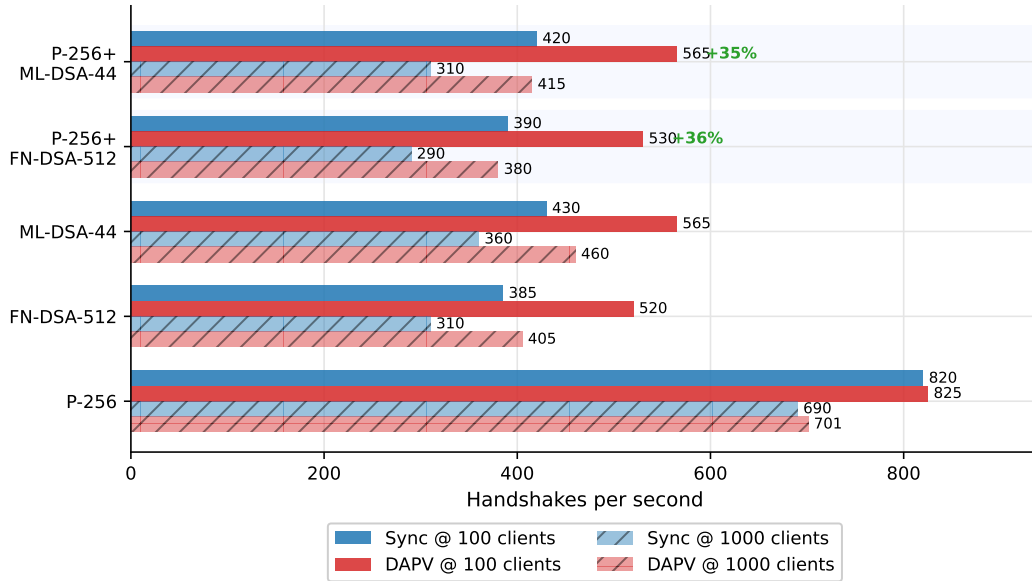


Figure 4: Completed handshakes per second with 100 (solid) and 1000 (hatched) concurrent clients; client and server on the same host.

here. Report the Linux-testbed saving and  $\Delta t$ ; if the Windows timer resolution is the cause, the Linux values should be much smaller.]

This finding has a direct consequence that we state plainly: for a single hybrid signature on a desktop-class CPU, most of the benefit we measure could also be obtained by optimising the synchronous path. The contribution of deferral itself is bounded by the synchronous PQ work that remains on the critical path, which is small in our configuration and larger in the cases listed in Section 4.6.

### 6.3. Completed-handshake rate

*Takeaway: with client and server on one host, DAPV raises the completed-handshake rate by 31–36%, consistently across concurrency and RTT.* Hybrid authentication lowers the rate from 820 to 390–420 handshakes/s at 100 clients and from 690 to 290–310 at 1000 clients (55–58%), relative to classical ECDSA (Figure 4). DAPV raises it by 35–36% at 100 clients, 31% at 500 clients (Table 3) and 31–34% at 1000 clients. Across RTT at 500

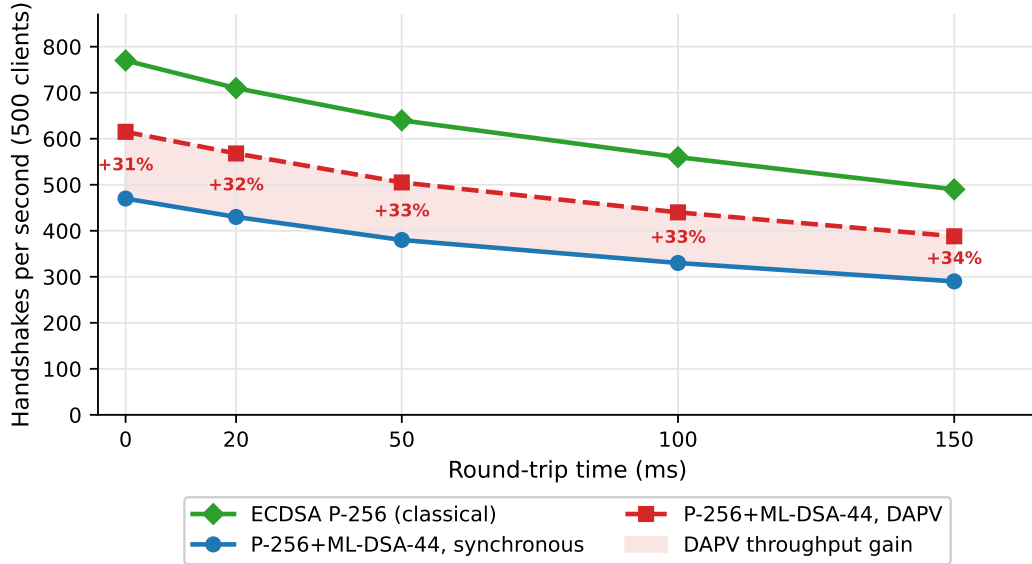


Figure 5: Completed handshakes per second versus RTT at 500 concurrent clients, P-256 + ML-DSA-44.

Table 2: Decomposition of  $\Delta t$  on idle hardware (30 runs; ms). “Crypto” is the verification call timed inside the worker; “Scheduling” is the remainder (thread creation, queueing, wake-up).

| Configuration      | Crypto | Scheduling | Total $\Delta t$ (mean) | 95% CI of mean |
|--------------------|--------|------------|-------------------------|----------------|
| P-256 + FN-DSA-512 | 0.14   | 18.76      | 18.9                    | [16.7, 21.1]   |
| P-256 + ML-DSA-44  | 0.10   | 21.10      | 21.2                    | [18.6, 23.8]   |

clients (Figure 5) the gain for P-256 + ML-DSA-44 stays between 31% and 34%.

Because DAPV changes only the client, it cannot reduce the server’s cryptographic work. Two mechanisms can raise the measured rate: the client-side load generator, which competes with the server for the same cores, spends less time blocked per handshake; and each connection holds server resources for a shorter time because the client **Finished** arrives earlier. Our single-host setup cannot separate them, and the first mechanism would not appear with clients on separate machines. The rate gain must therefore be read as a property of the combined client–server system, not as a server capacity gain. **[AUTHOR INPUT: Re-run with load generator and server on separate hosts (or at least pinned to disjoint cores) and report server-only throughput.]**

#### 6.4. CPU utilisation

*Takeaway: DAPV redistributes but does not reduce CPU work.* Peak host CPU utilisation is 35% for classical ECDSA and 79–85% for the hybrid configurations, with and without DAPV (Table 3); the same operations execute in both cases, but under DAPV the PQ verification runs on worker threads concurrently with other connections.

#### 6.5. Provisional window $\Delta t$

*Takeaway:  $\Delta t$  is set by scheduling, not cryptography, and grows under load.* Table 2 shows that the verification itself is under 1% of  $\Delta t$ ; the rest is scheduling. Under background CPU load (Figure 6), the 95th percentile of  $\Delta t$  grows from 23.8 ms (idle) to 34.2 ms at 50% and 58.7 ms at 80% load, i.e.,  $2.5\times$  the idle 95th percentile. Any data cap must therefore be calibrated against loaded, not idle,  $\Delta t$ . A bounded pool with pre-started workers (Algorithm 2) removes thread creation from  $\Delta t$  and should bring it close to the queueing plus verification time; we have not measured such an implementation, and the value it would achieve is a projection, not a result **[AUTHOR INPUT: if you implement and measure the pool, report the measured  $\Delta t$  here]**.

#### 6.6. Summary

Table 3 collects the measured values. Relative to classical ECDSA at 150 ms RTT, synchronous P-256 + ML-DSA-44 adds 25% latency and DAPV 12%; at 500 clients the completed-handshake rate is 39% below classical for the synchronous hybrid and 20% below with DAPV.

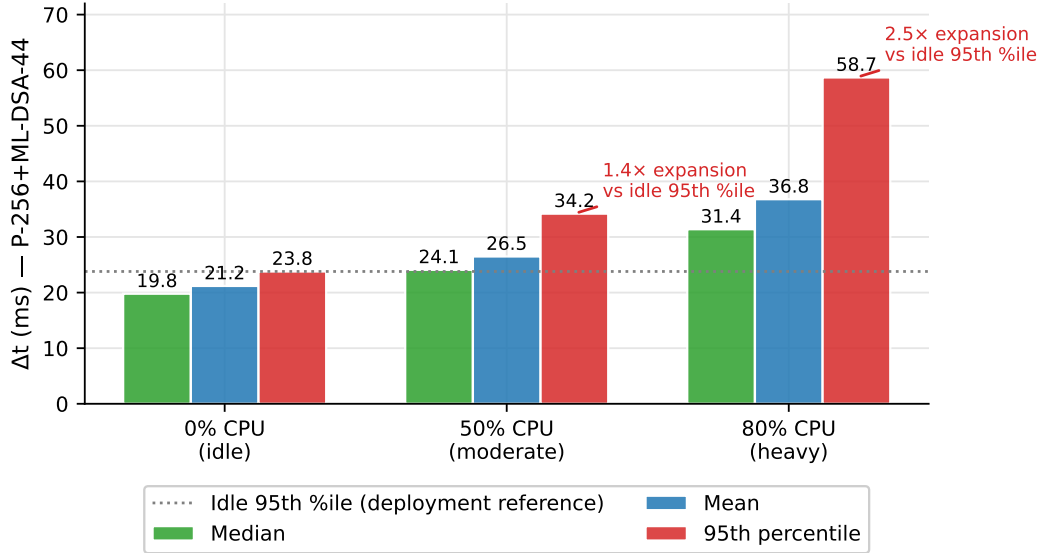


Figure 6:  $\Delta t$  for P-256 + ML-DSA-44 under 0%, 50% and 80% background CPU load: median, mean and 95th percentile.

Table 3: Summary of measured results. Rate and CPU at 500 concurrent clients and 0 ms RTT;  $\Delta t$  is the idle mean. Only measured values are shown.

| Configuration                   | Latency 0 ms<br>(ms) | Latency 150 ms<br>(ms) | Rate<br>(hs/s) | Peak CPU<br>(%) | $\Delta t$<br>(ms) |
|---------------------------------|----------------------|------------------------|----------------|-----------------|--------------------|
| ECDSA P-256                     | 15.2                 | 165                    | 770            | 35              | —                  |
| FN-DSA-512, synchronous         | 30.1                 | 180                    | 380            | 76              | —                  |
| ML-DSA-44, synchronous          | 41.2                 | 191                    | 390            | 82              | —                  |
| P-256 + FN-DSA-512, synchronous | 48.5                 | 199                    | 440            | 85              | —                  |
| P-256 + FN-DSA-512, DAPV        | 34.2                 | 179                    | 575 (+31%)     | 85              | 18.9               |
| P-256 + ML-DSA-44, synchronous  | 55.6                 | 206                    | 470            | 79              | —                  |
| P-256 + ML-DSA-44, DAPV         | 34.9                 | 185                    | 615 (+31%)     | 79              | 21.2               |

### 6.7. Threats to validity

- **Co-location.** Client and server share a host, so they compete for CPU and the rate measurements include client-side effects (Section 6.3). Traffic between them does not cross a physical network interface.
- **Emulated network.** On the loopback interface, the MTU is 64 KB and segmentation offload is typical, so segment counts and congestion-window behaviour differ from those on a real path; delay-only emulation without loss or bandwidth limits does not exercise loss recovery. Our conclusions about  $\delta k$  apply to the emulated setting; in particular, we do not claim to have measured the effect of MTU-sized segmentation.
- **Certificate chain.** A single-certificate chain understates the size and the number of verifications of real deployments, which typically send a leaf and one intermediate (and embed SCTs). Longer chains would increase both the size cost (possibly making  $\delta k > 0$ ) and the PQ work that DAPV can defer.
- **Timer resolution.** Several reported durations are of the same order as the Windows timer period (Section 6.2).
- **Prototype concurrency model.** One thread per job; the recommended bounded pool was not evaluated.

[AUTHOR INPUT: Where you have new data from separate hosts, real NICs or longer chains, report it and shorten this list accordingly.]

## 7. Security Analysis

The analysis below is a risk characterisation, not a proof. A formal analysis of the provisional state—e.g., in Tamarin, extending existing TLS 1.3 models [35]—is future work.

Table 4: Bandwidth term  $B \Delta t/8$  of Equation (3) for measured  $\Delta t$  values (decimal units). The last column is a hypothetical  $\Delta t$  of 1 ms, not a measured value.

| Link rate  | Measured $\Delta t$ |         |                         | Hypothetical |
|------------|---------------------|---------|-------------------------|--------------|
|            | 18.9 ms             | 21.2 ms | 58.7 ms (p95, 80% load) | 1 ms         |
| 1 Mbit/s   | 2.4 kB              | 2.7 kB  | 7.3 kB                  | 125 B        |
| 100 Mbit/s | 236 kB              | 265 kB  | 734 kB                  | 12.5 kB      |
| 1 Gbit/s   | 2.4 MB              | 2.7 MB  | 7.3 MB                  | 125 kB       |
| 10 Gbit/s  | 24 MB               | 27 MB   | 73 MB                   | 1.25 MB      |

### 7.1. Threat model

The network adversary controls the channel (Dolev–Yao). We distinguish (i) a *classical* adversary, who cannot forge ECDSA or PQ signatures but might hold a fraudulently issued classical credential or exploit a flaw in the classical component, and (ii) a *CRQC* adversary, who can forge ECDSA. Against (i), DAPV provides the same final guarantee as synchronous hybrid verification, but data sent during  $\Delta t$  reaches a peer authenticated only classically. Against (ii), the provisional state offers no protection: DAPV is appropriate only while CRQCs capable of real-time forgery are not a practical threat, and deployments must reassess it as that changes.

### 7.2. Leakage bound

The data the client can send before a failed PQ check tears down the connection is bounded by the link rate and by the TCP congestion window:

$$L_{\max} = \min\left(\frac{B \Delta t}{8}, W(\Delta t)\right), \quad (3)$$

where  $B$  is the bottleneck bandwidth in bit/s and  $W(\Delta t)$  is the amount the client’s congestion controller allows in flight during  $\Delta t$  on a new connection (about 14.6 KB per RTT at the initial window [16]). When  $\Delta t < \text{RTT}$ ,  $W(\Delta t)$  is at most one initial window, so the exposure is limited to about 14.6 KB irrespective of link speed; the bandwidth term dominates only on low-RTT paths. Table 4 gives the bandwidth term; the application cap  $D_{\text{policy}}$  bounds both.

*Choice of  $D_{\text{policy}}$ .* The cap is a policy parameter, not a quantity our measurements determine. Its correct value is the amount of data the application is willing to reveal to a peer that has passed only the classical check, which depends on the content, not on  $\Delta t$ . What  $\Delta t$  determines is how often the cap is reached and hence how often data must be buffered. As a heuristic default we suggest 8 KB: it is below one initial congestion window, so on WAN paths the cap rather than TCP is the binding limit; it is roughly the bandwidth term at 1 Mbit/s for the loaded 95th-percentile  $\Delta t$  (7.3 kB), so on slow links the cap is rarely reached; and it accommodates typical request headers and small requests. Applications should lower it, or set it to zero, for any content they consider sensitive.

### 7.3. Attack vectors and mitigations

*Impersonation with a valid classical credential.* An adversary holding a fraudulently obtained classical key for the server passes step 4 and receives up to  $L_{\max}$  bytes before step 8 terminates the session. Mitigation:  $D_{\text{policy}}$  and the restriction to non-sensitive content.

*Component stripping.* Replacing  $\sigma_P$  with an invalid value causes the server **Finished** check to fail (transcript binding); presenting a classical-only credential is rejected because the client requires the hybrid algorithm. Either way no session is promoted.

*Replay.* A **CertificateVerify** from another session does not verify against this session’s transcript hash, which includes both parties’ fresh key shares.

*Ticket inheritance.* Addressed by holding tickets until promotion (Section 4.3).

*Worker exhaustion (DAPV-specific).* In mutual-TLS or other settings where the verifier accepts many handshakes—e.g., a server verifying hybrid client certificates with DAPV—an adversary can send a valid  $\sigma_E$  with an invalid  $\sigma_P$  on many connections, each occupying a worker until its job fails. With one thread per job this exhausts memory and threads. Mitigations: a bounded pool with bounded queue and synchronous fall-back (Algorithm 2), which degrades DAPV to synchronous verification under attack instead of failing; and per-source rate limiting. Dispatching only after  $\sigma_E$  succeeds (Algorithm 1) means the attack requires a valid classical signature per connection.

Table 5: Positioning relative to prior work.

| Work                 | Cost addressed             | Approach                                                                  |
|----------------------|----------------------------|---------------------------------------------------------------------------|
| [19, 17, 18, 29, 20] | size and computation       | measurement; no mitigation                                                |
| [21, 22, 23]         | size (chain)               | omit or change certificates in the chain                                  |
| [24, 30, 25]         | size (handshake signature) | KEM-based implicit authentication                                         |
| [32]                 | computation (throughput)   | asynchronous offload; connection waits for result                         |
| This work            | computation (latency)      | defer PQ verification past handshake completion under a provisional state |

#### 7.4. Application-layer interface

Enforcing  $D_{\text{policy}}$  requires the TLS library to expose the state. We propose the following minimal OpenSSL-style interface **[AUTHOR INPUT: state whether this interface is implemented in the prototype or is a proposal]**:

```
#define SSL_DAPV_STATE_NONE      0  /* not a DAPV session      */
#define SSL_DAPV_STATE_PROVISIONAL 1 /* sigma_E ok, sigma_P pending */
#define SSL_DAPV_STATE_FULL_AUTH 2  /* sigma_E and sigma_P ok    */
#define SSL_DAPV_STATE_PQC_FAILED 3 /* sigma_P failed; closed    */

int SSL_get_dapv_state(const SSL *ssl);
void SSL_CTX_set_dapv_callback(SSL_CTX *ctx,
    void (*cb)(SSL *ssl, int new_state, void *arg), void *arg);
```

The callback is invoked on each state change, allowing an application to release buffered data on promotion and to abort pending work on failure; `SSL_write` can enforce  $D_{\text{policy}}$  by returning a retry-later indication while the session is provisional and the cap is reached.

## 8. Deployment Considerations

*Where DAPV fits.* DAPV suits clients that (i) can tolerate a bounded interval of classical-strength authentication for their first bytes, and (ii) perform enough synchronous PQ work per handshake for its removal to matter: long PQ chains, multi-algorithm credentials, SLH-DSA, or devices without vector units. Browsers fetching public content, API clients sending idempotent requests, and edge proxies are examples. Clients that immediately send credentials, payment data or personal data should verify synchronously or set  $D_{\text{policy}} = 0$  for such requests.

*Operational requirements.* A bounded worker pool; exposure of the provisional state to applications; monitoring of  $\Delta t$  (mean and 95th percentile), PQ failure rate and provisional-session count; calibration of any cap against  $\Delta t$  under production load; and the ticket rules of Section 4.3.

*Device classes.* On devices without vector units the verification itself becomes the dominant term of  $\Delta t$ , increasing both the benefit of deferral and the exposure window. We have not measured such devices; published microcontroller benchmarks [42] suggest verification times of milliseconds rather than tenths of a millisecond, and a deployment there should re-measure  $\Delta t$  before choosing  $D_{\text{policy}}$ .

*Standardisation.* Because DAPV changes no messages, it needs no TLS extension to interoperate. What would benefit from standardisation is guidance on the provisional semantics—ticket handling, permitted data and alerting—for implementations that choose to defer, analogous to the guidance that accompanied TLS False Start [33].

## 9. Comparison with Prior Work

Table 5 contrasts DAPV with prior work by target and method. Numerical comparison across studies is not meaningful because testbeds, chains and network conditions differ.

## 10. Conclusion

We studied deferring the PQ component of hybrid TLS 1.3 server authentication past handshake completion. DAPV requires no protocol change and is defined by the semantics of a provisional client state: classical authentication at completion, promotion on PQ success, teardown on failure, a data cap, and tickets withheld until promotion. In our prototype and single-host testbed, it lowers handshake latency by an RTT-independent 14–21 ms and raises the completed-handshake rate by 31–36%. Our decomposition shows that these gains exceed the cost of PQ arithmetic by two orders of magnitude, so for a single signature on a modern desktop CPU they reflect the cost of the prototype’s synchronous path rather than an intrinsic benefit of deferral. Deferral is most relevant where synchronous PQ work is genuinely large: multi-certificate chains, multi-algorithm credentials, slow schemes and constrained devices. The exposure it creates is bounded by the data cap and, on WAN paths, by the TCP initial window.

### *Future work*

1. Measurements with separate client and server hosts over real network interfaces, 2–3-certificate chains and X25519MLKEM768 key exchange.
2. Multi-algorithm hybrids and SLH-DSA, where several independent verifications can be parallelised across workers.
3. A bounded-pool implementation and its measured  $\Delta t$ .
4. A formal (e.g., Tamarin) model of the provisional state.
5. Server-side deferral for hybrid client certificates in mutual TLS.
6. Evaluation on ARM and microcontroller platforms.

## Acknowledgements

The authors thank the Open Quantum Safe project and the OpenSSL community for the open-source infrastructure used in this work.

## Declaration of Competing Interest

The authors declare no competing financial interests or personal relationships that could have influenced the work reported in this paper.

## References

- [1] P. W. Shor, Algorithms for quantum computation: Discrete logarithms and factoring, Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS) (1994) 124–134 [doi:10.1109/SFCS.1994.365700](#).
- [2] N. Alnahawi, J. Müller, J. Oupicky, A. Wiesmaier, A comprehensive survey on post-quantum TLS, IACR Communications in Cryptology 1 (2) (2024) 41–71.
- [3] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, D. Stebila, Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3, IETF Internet-Draft [draft-ietf-tls-ecdhe-mlkem](#), work in progress (2026).
- [4] National Institute of Standards and Technology, FIPS 203: Module-lattice-based key-encapsulation mechanism standard (MLKEM), Tech. rep., NIST (2024). [doi:10.6028/NIST.FIPS.203](#).
- [5] K. Kwiatkowski, L. Valenta, The TLS post-quantum experiment, Cloudflare Blog, <https://blog.cloudflare.com/the-tls-post-quantum-experiment/> (Oct. 2019).
- [6] M. Mosca, Cybersecurity in an era with quantum computers: Will we be ready?, IEEE Security & Privacy 16 (5) (2018) 38–41. [doi:10.1109/MSP.2018.3761723](#).
- [7] U.S. National Security Agency (NSA), Commercial national security algorithm suite and quantum computing roadmap, Tech. rep., NSA, <https://www.nsa.gov/Press-Room/News-Highlights/Article/Article/3215760> (2022).
- [8] European Union Agency for Cybersecurity (ENISA), Post-quantum cryptography: Preparing for the quantum era, Tech. rep., ENISA, <https://www.enisa.europa.eu/publications/post-quantum-cryptography> (2023).
- [9] U.K. National Cyber Security Centre (NCSC), Next steps in preparing for post-quantum cryptography, Tech. rep., NCSC, <https://www.ncsc.gov.uk/whitepaper/next-steps-preparing-for-post-quantum-cryptography> (2024).
- [10] National Institute of Standards and Technology, FIPS 204: Module-lattice-based digital signature standard (ML-DSA), Tech. rep., NIST, <https://doi.org/10.6028/NIST.FIPS.204> (2024).
- [11] National Institute of Standards and Technology, FIPS 205: Stateless hash-based digital signature standard (SLH-DSA / sphincs+), Tech. rep., NIST, <https://doi.org/10.6028/NIST.FIPS.205> (2024).
- [12] National Institute of Standards and Technology, FIPS 206 (initial public draft): FFT over NTRU-lattice-based digital signature algorithm (FN-DSA), Tech. rep., NIST, draft standard. AUTHORS: verify status, date and URL before submission. (2025).
- [13] N. Bindel, U. Herath, M. McKague, D. Stebila, Transitioning to a quantum-resistant public key infrastructure, in: T. Lange, T. Takagi (Eds.), Post-Quantum Cryptography (PQCrypto 2017), Vol. 10346 of Lecture Notes in Computer Science, Springer, Cham, 2017, pp. 384–405, provides nesting-based hybrid signature construction with composable unforgeability guarantees; differs from the concatenation construction used in our prototype.
- [14] N. Bindel, B. Hale, A note on hybrid signature schemes, Cryptology ePrint Archive, Paper 2023/423, <https://eprint.iacr.org/2023/423> (2023).
- [15] M. Ounsworth, J. Gray, M. Pala, J. Klaussner, S. Fluhrer, Composite ML-DSA for use in X.509 public key infrastructure, IETF Internet-Draft [draft-ietf-lamps-pq-composite-sigs](#), work in progress (2026).

- [16] J. Chu, N. Dukkupati, Y. Cheng, M. Mathis, Increasing TCP's initial window, RFC 6928, IETF (2013). doi:10.17487/RFC6928.
- [17] D. Sikeridis, P. Kampanakis, M. Devetsikiotis, Post-quantum authentication in TLS 1.3: A performance study, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), 2020. doi:10.14722/ndss.2020.24203.
- [18] D. Sikeridis, P. Kampanakis, M. Devetsikiotis, Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the 16th International Conference on Emerging Networking Experiments and Technologies (CoNEXT'20), ACM, 2020, pp. 149–156. doi:10.1145/3386367.3431305.
- [19] C. Paquin, D. Stebila, G. Tamvada, Benchmarking post-quantum cryptography in TLS, in: J. Ding, J.-P. Tillich (Eds.), Post-Quantum Cryptography, Vol. 12100 of Lecture Notes in Computer Science, Springer, Cham, 2020, pp. 72–91. doi:10.1007/978-3-030-44223-1\_5.
- [20] J. A. Montenegro, R. Rios, J. Lopez-Cerezo, A performance evaluation framework for post-quantum TLS, Future Generation Computer Systems 175 (2026) 108062. doi:10.1016/j.future.2025.108062.
- [21] P. Kampanakis, M. Kallitsis, Faster post-quantum TLS handshakes without intermediate CA certificates, in: Cyber Security, Cryptology, and Machine Learning (CSCML 2022), Vol. 13301 of Lecture Notes in Computer Science, Springer, 2022. doi:10.1007/978-3-031-07689-3\_25.
- [22] D. Sikeridis, S. Huntley, D. Ott, M. Devetsikiotis, Intermediate certificate suppression in post-quantum TLS: An approximate membership querying approach, in: Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies (CoNEXT), ACM, 2022. doi:10.1145/3555050.3569127.
- [23] S. Paul, Y. Kuzovkova, N. Lahr, R. Niederhagen, Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3, in: Proceedings of the ACM Asia Conference on Computer and Communications Security (ASIA CCS), 2022, pp. 727–740. doi:10.1145/3488932.3497755.
- [24] P. Schwabe, D. Stebila, T. Wiggers, Post-quantum TLS without handshake signatures, in: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), 2020, pp. 1461–1480. doi:10.1145/3372297.3423350.
- [25] T. Wiggers, S. Celi, P. Schwabe, D. Stebila, N. Sullivan, KEM-based authentication for TLS 1.3, IETF Internet-Draft **draft-celi-wiggers-tls-authkem**, <https://datatracker.ietf.org/doc/draft-celi-wiggers-tls-authkem/> (Work in Progress). This draft specifies KEM-based authentication; hybrid signature design is covered by separate IETF work items. (2024).
- [26] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, D. Stehlé, CRYSTALS-Dilithium: A lattice-based digital signature scheme, IACR Transactions on Cryptographic Hardware and Embedded Systems 2018 (1) (2018) 238–268. doi:10.13154/tches.v2018.i1.238-268.
- [27] N. Bindel, B. Hale, D. Connolly, F. Driscoll, Hybrid signature spectrums, IETF Internet-Draft **draft-ietf-pquip-hybrid-signature-spectrums**, work in progress. AUTHORS: verify author list and year.
- [28] E. Rescorla, The transport layer security (TLS) protocol version 1.3, RFC 8446, IETF (2018). doi:10.17487/RFC8446.
- [29] M. Sosnowski, F. Wiedner, E. Hauser, L. Steger, D. Schoinianakis, S. Gallenmüller, G. Carle, The performance of post-quantum TLS 1.3, in: Companion of the 19th International Conference on Emerging Networking Experiments and Technologies (CoNEXT Companion '23), ACM, 2023, doi = 10.1145/3624354.3630585.
- [30] P. Schwabe, D. Stebila, T. Wiggers, More efficient post-quantum KEMTLS with pre-distributed public keys, in: Computer Security – ESORICS 2021, Vol. 12972 of Lecture Notes in Computer Science, Springer, 2021, pp. 3–22. doi:10.1007/978-3-030-88418-5\_1.
- [31] R. Gonzalez, T. Wiggers, KEMTLS vs. post-quantum TLS: Performance on embedded systems, in: Proceedings of the International Conference on Security, Privacy and Applied Cryptographic Engineering (SPACE), 2022, <https://kemtls.org/publication/kemtls-embedded/>.
- [32] X. Hu, C. Wei, J. Li, B. Will, P. Yu, L. Gong, H. Guan, QTLS: High-performance TLS asynchronous offload framework with Intel QuickAssist technology, in: Proceedings of the 24th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP), 2019, pp. 158–172. doi:10.1145/3293883.3295705.
- [33] A. Langley, N. Modadugu, B. Moeller, Transport layer security (TLS) false start, RFC 7918, IETF (2016). doi:10.17487/RFC7918.
- [34] C. Cremers, M. Horvat, S. Scott, T. van der Merwe, Automated analysis and verification of TLS 1.3: 0-RTT, resumption and delayed authentication, in: Proceedings of the IEEE Symposium on Security and Privacy (S&P), 2016, pp. 470–485. doi:10.1109/SP.2016.35.
- [35] C. Cremers, M. Horvat, J. Hoyland, S. Scott, T. van der Merwe, A comprehensive symbolic analysis of TLS 1.3, in: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017, pp. 1773–1788. doi:10.1145/3133956.3134063.
- [36] M. Braithwaite, Experimenting with post-quantum cryptography, Google Security Blog, <https://security.googleblog.com/2016/07/experimenting-with-post-quantum.html> (Jul. 2016).
- [37] A. Langley, CECQP1 results, Imperial Violet Blog, <https://www.imperialviolet.org/2016/11/28/cecpq1.html> (Nov. 2016).
- [38] A. Langley, CECQP2, Imperial Violet Blog, <https://www.imperialviolet.org/2018/12/12/cecpq2.html> (Dec. 2018).
- [39] A. Langley, Quantum-resistant authentication in TLS, Chromium Blog, <https://blog.chromium.org/2023/09/quantum-resistant-authentication-in-tls.html> (Sep. 2023).
- [40] T. Reddy, H. Tschofenig, Post-quantum cryptography recommendations for TLS-based applications, IETF Internet-Draft **draft-ietf-uta-pqc-app-00**, <https://datatracker.ietf.org/doc/draft-ietf-uta-pqc-app/> (Work in Progress) (Sep. 2025).
- [41] D. Stebila, M. Mosca, Post-quantum key exchange for the internet and the open quantum safe project, in: R. Avanzi, H. Heys (Eds.), Selected Areas in Cryptography (SAC 2016), Vol. 10532 of Lecture Notes in Computer Science, Springer, Cham, 2017, pp. 1–24, oQS benchmarks available at <https://openquantumsafe.org/>.
- [42] M. J. Kannwischer, J. Rijneveld, P. Schwabe, K. Stoffelen, pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4, in: Second NIST PQC Standardization Conference, 2019.